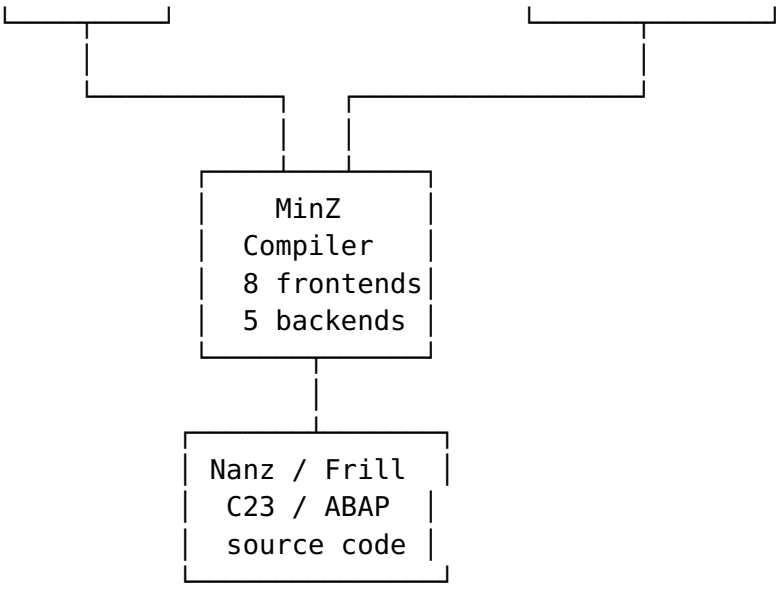
One Compiler, 50 Years: From Z80 (1976) to NVIDIA GPU (2026)

MinZ Project

2026-03-27

One Compiler, 50 Years

Same source code. Same frontend. Five backends. Z80 to GPU.



The Demo

```
fun double(x: u8) -> u8 {
    return x + x
}
```

This function compiles to:

Z80 (1976)

```
double:
    ADD A, A    ; 1 byte, 4 T-states
    RET         ; 1 byte, 10 T-states
; Total: 2 bytes, 14 T-states
```

CUDA (NVIDIA GPU)

```
__global__ void kernel(uint8_t* in, uint8_t* out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = in[i] + in[i];
}
// All 256 inputs computed in parallel: ~0.001ms
```

OpenCL (AMD, Intel, any GPU)

```
        __kernel void kernel(__global uchar* in, __global uchar* out, int
n) {
            int i = get_global_id(0);
```

```
            if (i < n) out[i] = in[i] + in[i];
        }
```

Vulkan (GLSL → SPIR-V)

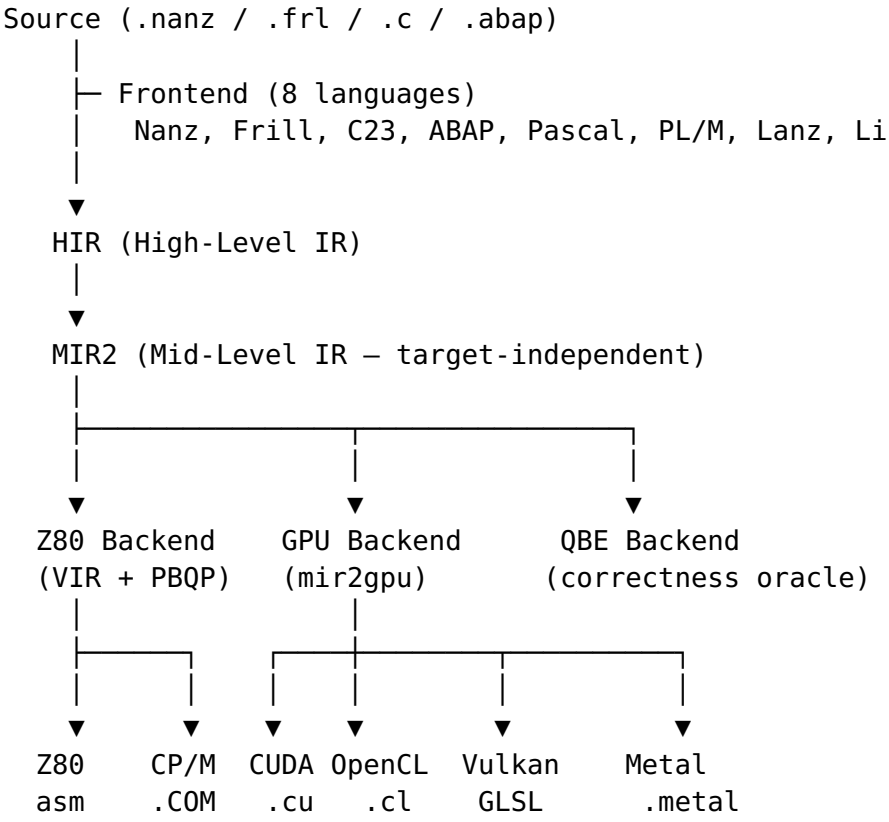
```
        layout(set=0, binding=0) buffer In  { uint data_in[];  };
        layout(set=0, binding=1) buffer Out { uint data_out[]; };
        void main() {
            uint i = gl_GlobalInvocationID.x;
            data_out[i] = (data_in[i] + data_in[i]) & 0xFF;
        }
```

Metal (Apple Silicon)

```
kernel void func(device uint8_t* in, device uint8_t* out,
                 uint i [[thread_position_in_grid]]) {
    out[i] = in[i] + in[i];
}
```

One function. Five targets. 50 years of hardware.

How It Works



The key insight: **MIR2 is target-independent.** It has ~30 opcodes (add, sub, mul, cmp, branch, call, return). Any backend that can lower these opcodes to its target can compile any MinZ program.

The GPU backend (`mir2gpu`) is **700 LOC total.** 95% is shared across all four GPU APIs. Only 5% is backend-specific: kernel qualifiers, thread ID syntax, parameter passing.

Why This Matters

1. GPU as Exhaustive Verification Oracle

Compile the same function to Z80 and GPU. Run all 256 inputs on GPU in parallel (~0.001ms). Run all 256 inputs on Z80 emulator (~0.1s). Compare outputs.

If they match → **mathematically proven correct** for the entire u8 domain.

This is not testing. This is **exhaustive proof.**

```
                GPU (parallel)           Z80 (sequential)
Input:     [0, 1, 2, ... 255]      [0, 1, 2, ... 255]
Output:    [0, 2, 4, ... 254]      [0, 2, 4, ... 254]
                     ↓                        ↓
              MATCH → PROVEN CORRECT ✓
```

For u16 functions: GPU tests all 65,536 inputs in parallel. For two-argument u8 functions: 256×256 = 65,536 combinations. Still instant on GPU.

2. Cross-Architecture Verification

The same MIR2 function is lowered by completely independent backends: - Z80 backend: VIR (Z3 SMT solver) + PBQP heuristic - CUDA backend: direct C translation - OpenCL backend: direct C translation - Vulkan backend: GLSL compute shader

If all four backends produce the same output for all inputs, the MIR2 semantics are correct. Each backend is a **witness** to the others.

3. Functional Languages on GPU

Frill (ML-style) compiles through the same pipeline:

```
type Entity = Player | Enemy | Bullet | Coin | Wall

let is_solid (e : u8) : u8 =
  match e with
  | Player -> 0  | Enemy -> 1  | Bullet -> 0
  | Coin   -> 0  | Wall  -> 1
  end
```

This compiles to: - **Z80:** 175 bytes, pattern match → conditional jumps - **CUDA:** parallel evaluation of all entity types

ADTs, pattern matching, pipe operators — all on GPU. Not through a VM or interpreter. **Native compiled code.**

4. 8 Frontend Languages × 5 Backends = 40 Combinations

Frontend	Z80	CUDA	OpenCL	Vulkan	Metal
Nanz (Swift-like)	✅	✅	✅	✅	✅*
Frill (ML)	✅	✅	✅	✅	✅*
C23	✅	✅	✅	✅	✅*
ABAP	✅	✅	✅	✅	✅*
Pascal	✅	✅	✅	✅	✅*
PL/M	✅	✅	✅	✅	✅*
Lanz	✅	✅	✅	✅	✅*
Lizp	✅	✅	✅	✅	✅*

*Metal: generates valid MSL, needs Apple Silicon for testing.

Any function written in any of the 8 languages can run on any of the 5 backends. The MIR2 intermediate representation is the universal bridge.

The Numbers

Metric	Value
Frontend languages	8

Metric	Value
Backend targets	5 (Z80 + CUDA + OpenCL + Vulkan + Metal)
Year span	1976–2026 (50 years)
GPU backend LOC	700 (95% shared across 4 APIs)
CUDA verification	256/256 correct on real NVIDIA GPU
OpenCL verification	256/256 correct on real GPU
Z80 corpus asserts	1046
Z80 VIR codegen	-71% vs SDCC
GPU precomputed tables	83.6M register allocations, 501 arithmetic sequences

---

The Compiler Architecture

MinZ isn't a transpiler. It's a **real compiler** with:

- **Z3 SMT solver** for provably optimal register allocation
- **PBQP heuristic** fallback for complex functions
- **ISLE term rewriting** for instruction selection
- **Grace graph rewriting** for CFG optimization
- **GPU precomputed tables** (83.6M entries) for O(1) register allocation
- **RLCA sled** (9-byte multi-entry barrel shifter) for Z80 rotation
- **TSMC** (True Self-Modifying Code) for runtime optimization
- **#embed** (C23) for compile-time binary data inclusion
- **BCD packed decimal** types for COBOL/financial arithmetic

The Z80 backend alone is ~11K LOC. The entire compiler is ~90K LOC in Go.

What's Next

- **BCD arithmetic on GPU:** verify Z80 DAA sequences against GPU reference
- **FP16 soft-float:** GPU-precomputed mantissa tables
- **COBOL frontend:** PIC 9 types → BCD → DAA on Z80 / parallel on GPU

- **BASIC frontend:** The most iconic retro language, now compilable to GPU
- **WebGPU backend:** Run the same code in the browser

Try It

```
        # Z80
        git clone https://github.com/oisee/minz
        cd minz/minzc && go build -o mz ./cmd/minzc
        ./mz examples/frill/state_machine.frl -o out.a80    # 175 bytes

        # GPU (requires feat/mir2gpu branch)
        ./mz examples/nanz/01_hello.nanz --target=cuda -o out.cu
        nvcc out.cu -o gpu_test && ./gpu_test                # 256/256
correct
```

50 years. One compiler. Five backends. Zero compromises.

MinZ v0.23.0 — Birthday Marathon Release. "The compiler never fails. It only varies in how optimal the result is."